

Project Report

Formal project report, system design, implementation, testing, and deployment

Project	HAL Smart PDF Approval & QR Verification System
Purpose	A presentation-ready report suitable for project review, viva preparation, and portfolio documentation.
Stack	Spring Boot 3.5.16 · Java 21 · React/Vite · PostgreSQL · JWT · PDFBox · ZXing · Docker

Prepared as project documentation and presentation-ready reference material.

Contents

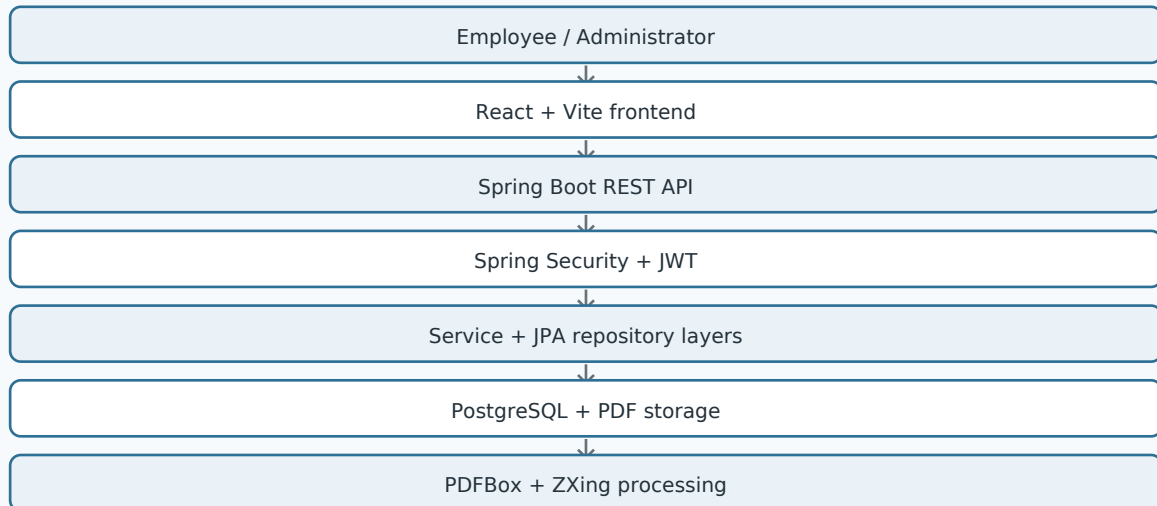
A structured reference for development, documentation, and interviews.

Section	Coverage
1. Executive Summary	System overview and purpose
2. Problem Statement	Workflow problems addressed
3. Objectives	Functional project goals
4. Requirements	Functional and non-functional requirements
5. Architecture	Application and deployment architecture
6. Database	Core entities and relationships
7. Modules	Responsibilities of major modules
8. Workflow	End-to-end approval lifecycle
9. Implementation	Important technologies and internals
10. Testing	Validation and observed results
11. Deployment	Docker, Vercel, Railway
12. Future Scope	Limitations and improvements
13. Conclusion	Project outcome

1. Executive Summary

The HAL PDF Approval System is a full-stack web application for submitting, reviewing, approving/rejecting, and verifying PDF documents. It combines a React/Vite frontend with a Spring Boot REST backend, PostgreSQL persistence, JWT authentication, PDFBox document processing, ZXing QR generation, and Docker support.

High-level system



2. Problem Statement

Manual document workflows can involve email attachments, paper signatures, unclear status, duplicated versions, and weak verification. A centralized digital workflow can make submission, review, status tracking, approval metadata, and verification more consistent.

3. Objectives

- Authenticate employees and administrators.
- Allow employees to upload PDF documents.
- Provide administrator review and approval/rejection.
- Record review comments and status.
- Generate QR-enabled approved PDFs.
- Persist core metadata in PostgreSQL.
- Provide a deployable full-stack application.

4. Requirements

Functional

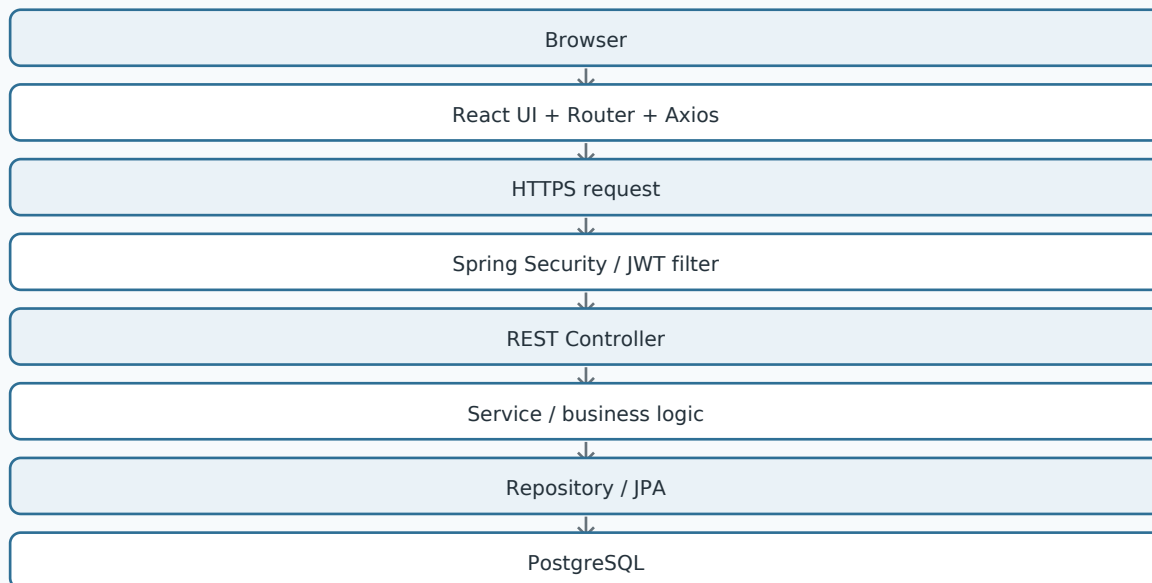
- Login with email/password.
- JWT-based authenticated sessions.
- PDF upload.
- Document listing and status visibility.
- Administrator approval/rejection.
- Comments on review decisions.
- Approved PDF download.
- QR generation during approval processing.

Non-functional

- BCrypt password hashing and JWT security.
- Layered and maintainable Spring Boot architecture.
- Docker-based local orchestration.
- Clear employee/admin frontend flows.
- Production configuration through environment variables.
- Storage architecture that can later move to object storage.

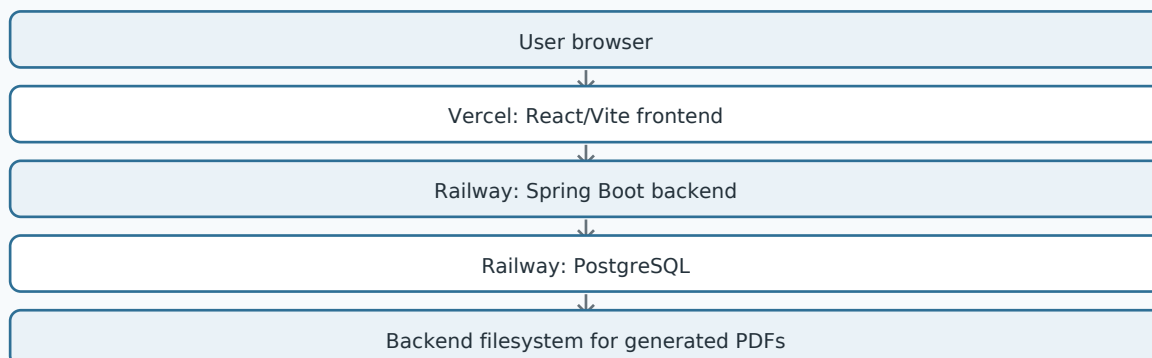
5. System Architecture

Application architecture



The PDF pipeline is handled on the backend: ZXing creates the QR image and PDFBox inserts it into the approved document.

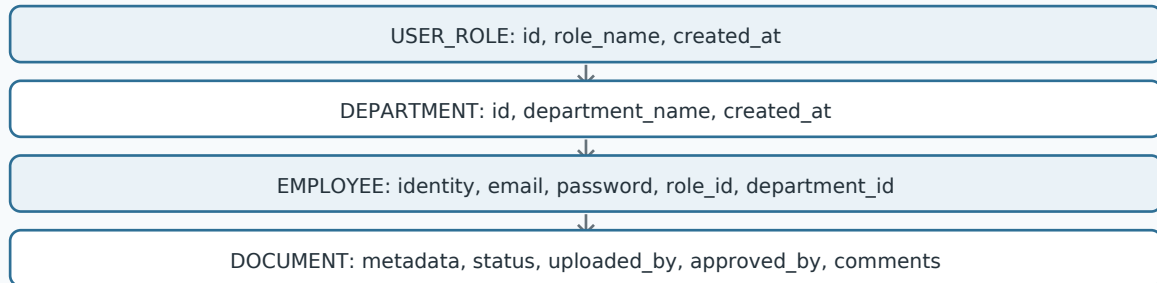
Production deployment



6. Database Design

The core model contains employees, roles, departments, and documents. Documents reference the employee who uploaded them and, when applicable, the employee who approved them.

Core relational model



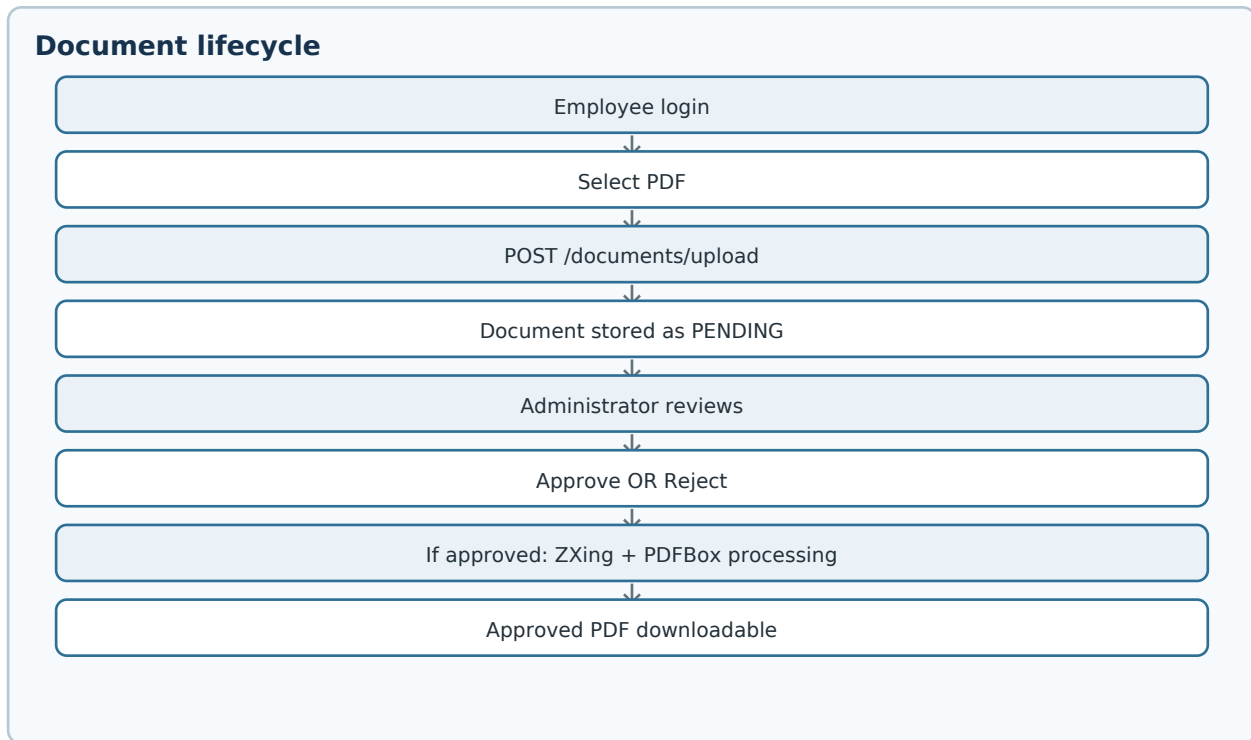
Key relationships

- Employee belongs to a department.
- Employee has a role.
- Document references uploader employee.
- Document can reference approver employee.
- Unique constraints protect employee email/employee ID and department name.

7. Module Design

Module	Responsibility
Authentication	Login, BCrypt verification, JWT generation
Security	JWT filter, CORS, endpoint/method protection
Employee	Identity, role, department data
Document	Upload, status, approval/rejection, download
PDF / QR	PDFBox modification and ZXing QR creation
Frontend	Routing, forms, dashboards, API calls

8. End-to-End Workflow



9. Implementation Highlights

Spring Boot layering

Controllers handle HTTP contracts, services coordinate business rules and workflows, and repositories provide persistence through Spring Data JPA.

JWT internals

JwtAuthenticationFilter examines the Authorization header. The JWT service validates/parses the token, and successful authentication is placed into Spring Security's SecurityContext.

PDFBox + ZXing

ZXing converts verification data into a QR image; PDFBox opens and modifies the PDF. Keeping this processing server-side keeps the browser focused on presentation and API interaction.

10. Testing

Test	Expected	Outcome
Build/test	Build succeeds; tests pass	Passed
Employee login	JWT returned	Passed
Admin login	JWT returned	Passed
PDF upload	PENDING record created	Passed
Admin approval	APPROVED state + processing	Passed
Approved download	QR-enabled PDF available	Passed

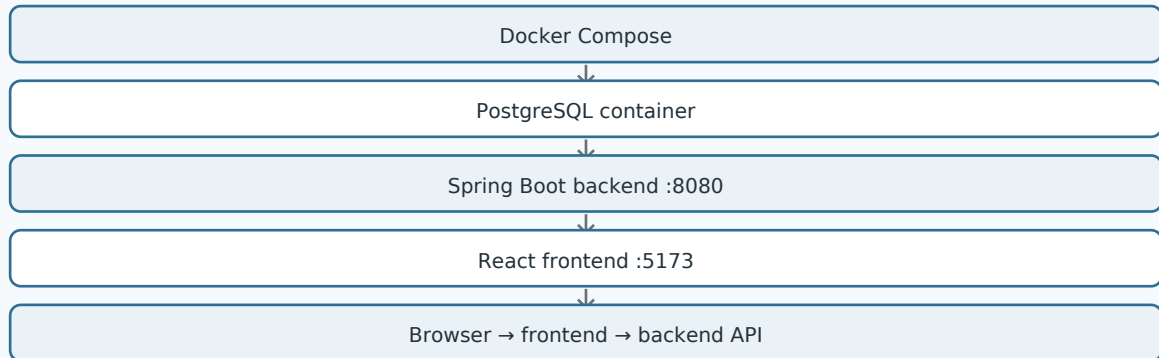
Production debugging lessons

- Production database initially needed schema initialization.
- JWT signing requires a strong non-empty secret.
- Vite frontend API URL is supplied at build time.
- SPA routing requires a rewrite so direct /login refreshes resolve correctly.

11. Deployment

Local

Docker Compose setup



Production

- Frontend deployed on Vercel.
- Backend deployed on Railway.
- PostgreSQL hosted on Railway.
- Environment variables provide database, JWT, CORS, and frontend API configuration.

12. Limitations & Future Scope

- Move PDF storage to durable object storage.
- Add a public QR verification endpoint.
- Introduce immutable audit events.
- Add email/in-app approval notifications.
- Add pagination, filtering, and search.
- Strengthen refresh-token/session management.
- Add CI/CD and broader integration tests.

13. Conclusion

The project demonstrates a complete full-stack workflow: authenticate → upload → review → approve/reject → generate QR-enabled PDF → download/verify. It connects frontend development, REST APIs, Spring Security, JPA/PostgreSQL, PDF processing, QR generation, Docker, and cloud deployment into one coherent system.

Presentation-ready summary

“I built a role-based PDF approval system using React and Spring Boot. Employees upload PDFs, administrators review and approve or reject them, and approved PDFs are processed with PDFBox and ZXing to add QR-based verification. PostgreSQL stores metadata, JWT secures the REST API, Docker supports local deployment, and the application is deployed with Vercel and Railway.”

End of Project Report